UNIVERSITY PARTNER

UNIVERSITY OF WOLVERHAMPTON

HERALD COLLEGE KATHMANDU

6CS012 - Artificial Intelligence & Machine Learning

Assignment 2 - Text Classification with RNN & Variant

Vehicle Type Recognition Using Convolutional Neural Networks

Student Name : Sushmita Shrestha

University ID : 2408996

Lecturer : Mr. Shiv Yadav

Tutor : Ms. Jinu Nyachhyol

Submission Date : May 10, 2026

Abstract

This report presents a fully paragraph-based explanation of the vehicle image classification code and its results. The project uses convolutional neural networks to classify images into six vehicle categories: Auto Rickshaws, Bikes, Cars, Motorcycles, Ships and Trains. The workflow covers data loading, preprocessing, augmentation, model construction, training, validation, testing and performance comparison. Several approaches were compared, including a baseline CNN, deeper CNN models trained with Adam and SGD, MobileNetV2 transfer learning and fine-tuned MobileNetV2. The baseline CNN achieved the strongest reported accuracy of 78.20%, while fine-tuned transfer learning reached 77.30% with shorter training time. Overall, the code demonstrates that reliable image classification depends not only on model depth, but also on data preparation, augmentation, regularization, optimizer selection and available computing resources.

Table of Contents

Table of Figure

Introduction

Image classification is an important area of computer vision because it allows a computer system to identify the object or category shown in an image. In this assignment, the code was developed to recognise different types of vehicles from image data. This kind of system can be useful in traffic monitoring, smart parking systems, transport management, vehicle counting and automated surveillance. The main aim of the project was to build a complete deep learning pipeline that could train and compare different CNN-based models for vehicle type recognition.

The report explains the code in a human-readable academic style. Instead of only showing output values, it describes why each stage was used and how each result should be understood. The code starts by preparing the dataset, then applies image augmentation, trains several models and evaluates them using accuracy, precision, recall, F1-score, confusion matrices and learning curves. This makes the project stronger because it shows both implementation and interpretation.

Dataset and Preprocessing

The dataset contains 4,768 RGB vehicle images collected from an online UCL dataset. The images are grouped into six classes: Auto Rickshaws, Bikes, Cars, Motorcycles, Ships and Trains. Before training, all images were resized to 224 by 224 pixels so that the models received inputs of the same shape. The image pixel values were also normalised to a suitable numerical range, which helps the model learn more smoothly and reduces the effect of very large raw pixel values.

The dataset was split into training and validation data. The training data was used by the models to learn visual patterns such as edges, shapes, textures and vehicle structures. The validation data was kept separate so that the model performance could be checked on unseen images during training. This separation is important because a model that performs well only on the training set may not work well on new images.

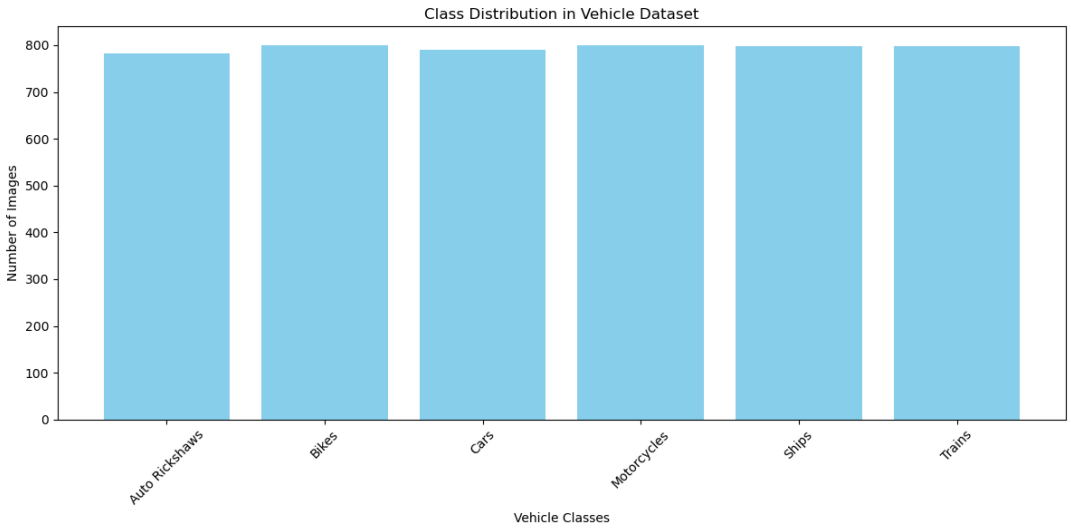

Figure 1: Class distribution in the vehicle dataset.

The class distribution figure shows that the dataset is reasonably balanced across the six vehicle categories. A balanced dataset is helpful because it gives the model a fair opportunity to learn each class. If one class had many more images than the others, the model could become biased towards that majority class.

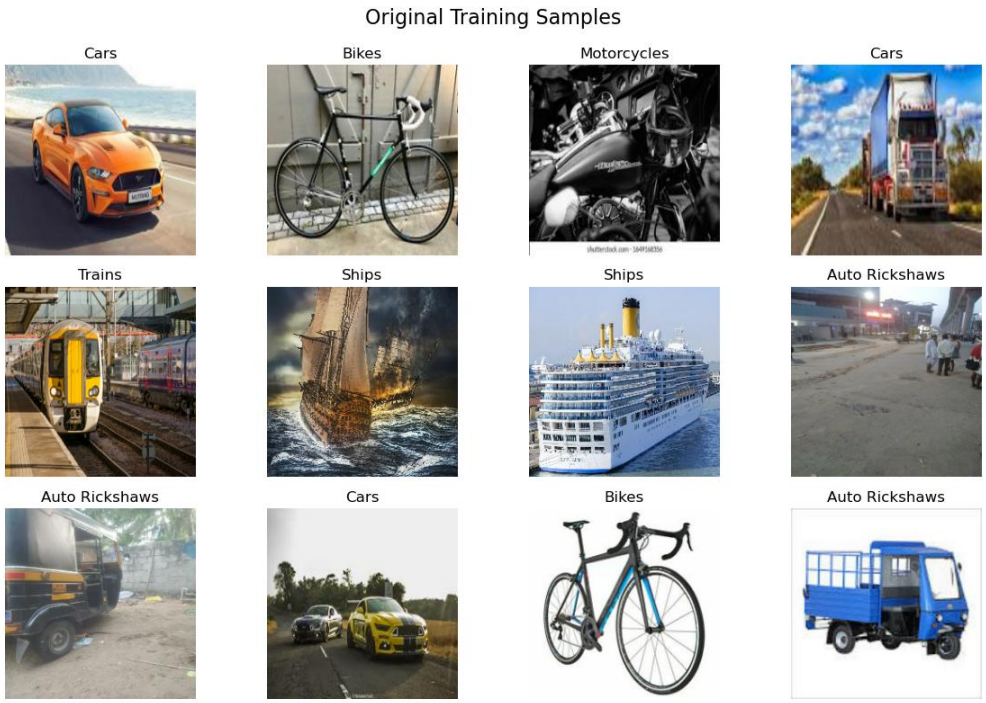

Figure 2: Original training samples from the vehicle dataset.

The original training images show that the dataset includes variation in backgrounds, lighting, object size, camera angle and image quality. These natural differences make the task more realistic. They also make the task more difficult because the model must learn meaningful vehicle features rather than memorising one fixed image pattern.

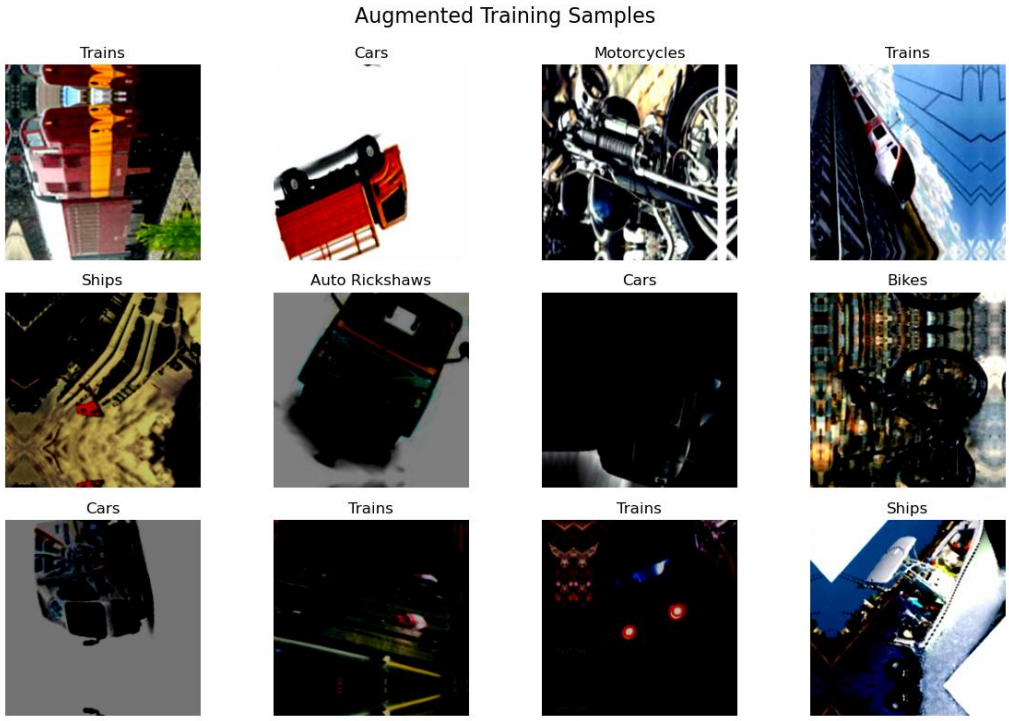

Figure 3: Augmented training samples created during preprocessing.

Data augmentation was applied to improve generalisation. The code used transformations such as flipping, rotation, zooming, translation, brightness changes, contrast adjustment, Gaussian noise and cropping. These transformations create new variations from the existing training images. As a result, the model becomes more robust because it learns to recognise vehicles even when they appear at different positions, angles or brightness levels.

Code Workflow

The code follows a structured workflow from data preparation to final model comparison. First, the required Python libraries were imported, including TensorFlow and Keras for deep learning,

NumPy and Pandas for numerical and data handling tasks, Matplotlib for visualisation and Scikit-learn for evaluation metrics. After that, important settings such as image size, batch size, random seed and dataset directory were defined. These settings helped keep the experiment organised and consistent.

After defining the settings, the images were loaded from their class folders and divided into training and validation sets. The preprocessing layer prepared the images for the neural networks, while the augmentation layer generated modified versions of training images. The code then built different models, compiled them using suitable loss functions and optimizers, trained them for several epochs and stored the training history. Finally, the code generated accuracy curves, loss curves, confusion matrices and a performance comparison table to support a fair evaluation.

Model Architecture and Methodology

The first model was a baseline convolutional neural network. It used three convolutional layers with 32, 64 and 128 filters. Each convolutional layer was followed by ReLU activation and max pooling so that the model could extract spatial image features and reduce feature-map size. After the convolutional blocks, the features were flattened and passed through dense layers before the final softmax output layer predicted one of the six vehicle classes. Dropout was included to reduce overfitting.

The second modelling approach was a deeper CNN. This model increased the number of convolutional layers and added Batch Normalization after convolution operations. Batch Normalization helped stabilise the learning process, while Dropout reduced the chance of memorising training images. The deeper CNN was trained using both Adam and SGD optimizers so that the effect of optimizer selection could be compared.

The third approach used transfer learning with MobileNetV2, a model that had already learned useful image features from the ImageNet dataset. In the feature extraction stage, MobileNetV2 was used as a frozen base model and a new classification head was added for the six vehicle classes. In the fine-tuning stage, some of the upper layers were unfrozen and trained again for the vehicle dataset. This allowed the model to adapt its learned features more closely to the assignment dataset.

Experiments and Results

The baseline CNN reached a reported validation accuracy of 78.20%. This result shows that a simple CNN can perform well when the dataset is prepared correctly and when the model is trained with a suitable structure. However, the learning curves indicate that the validation loss fluctuated, which suggests that the model still faced some generalisation difficulty.

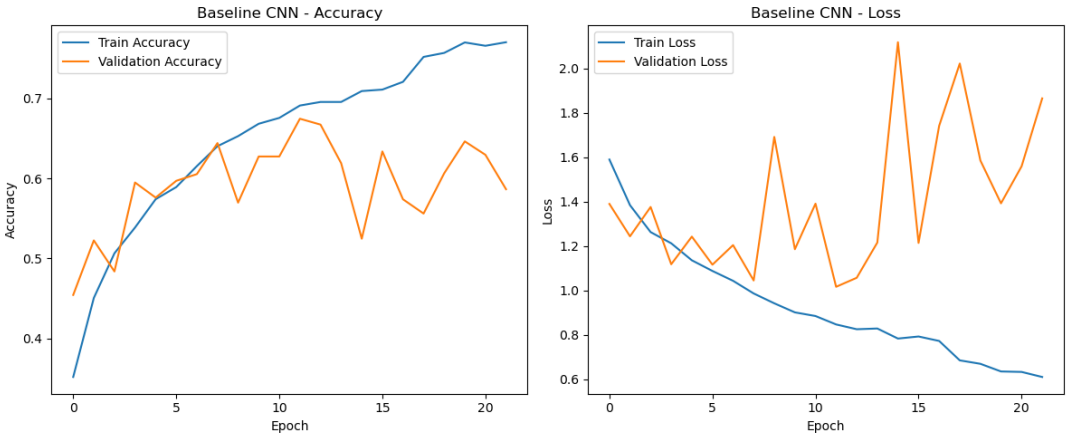

Figure 4: Accuracy and loss curves for the baseline CNN.

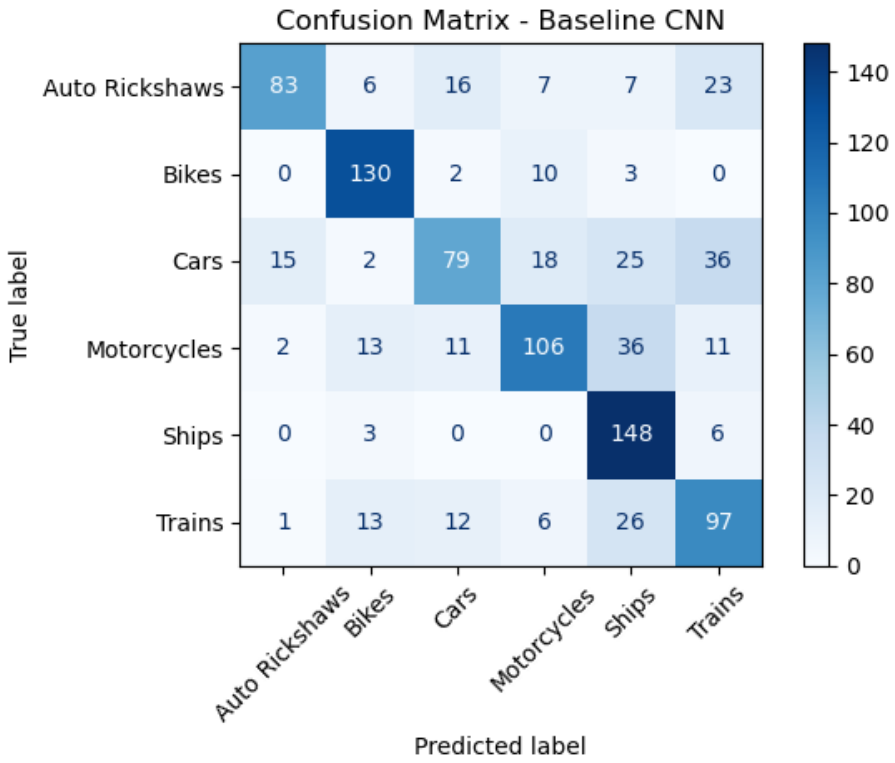

Figure 5: Confusion matrix for the baseline CNN.

The baseline confusion matrix shows that the model classified Bikes and Ships strongly, while Cars, Trains and Auto Rickshaws had more confusion. This is understandable because some vehicle classes may share visual features such as wheels, road backgrounds or similar object shapes. The confusion matrix is useful because it shows not only the final accuracy, but also where the model made mistakes.

The deeper CNN trained with Adam achieved a reported accuracy of 76.67%. The model converged smoothly and Adam helped it learn quickly in the early epochs. However, the deeper architecture required much more training time than the baseline CNN. This shows that increasing model depth can improve feature learning, but it also increases computational cost and does not always produce the highest accuracy.

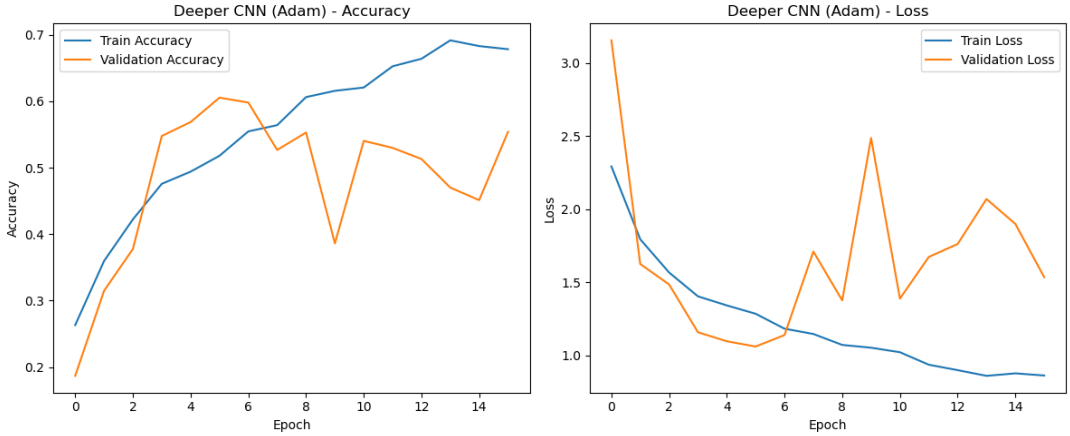

Figure 6: Accuracy and loss curves for the deeper CNN trained with Adam.

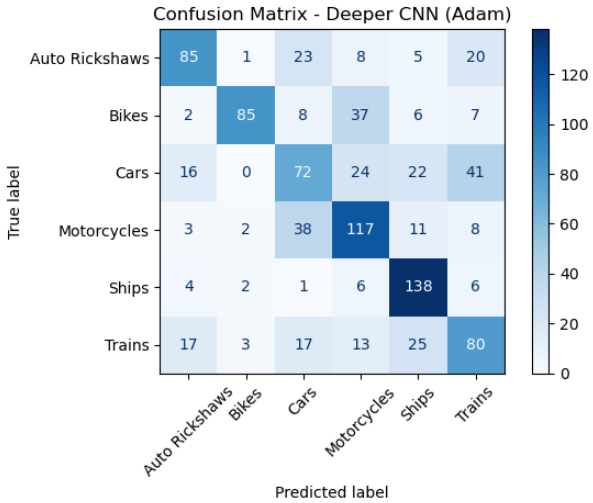

Figure 7: Confusion matrix for the deeper CNN trained with Adam

The deeper CNN trained with SGD reached a reported accuracy of 72.52%. Compared with Adam, SGD improved more slowly and required more epochs to reach stable performance. This result demonstrates that optimizer choice has a strong impact on convergence speed and final model quality. Adam was more effective for this dataset and model structure because it adapts learning rates during training.

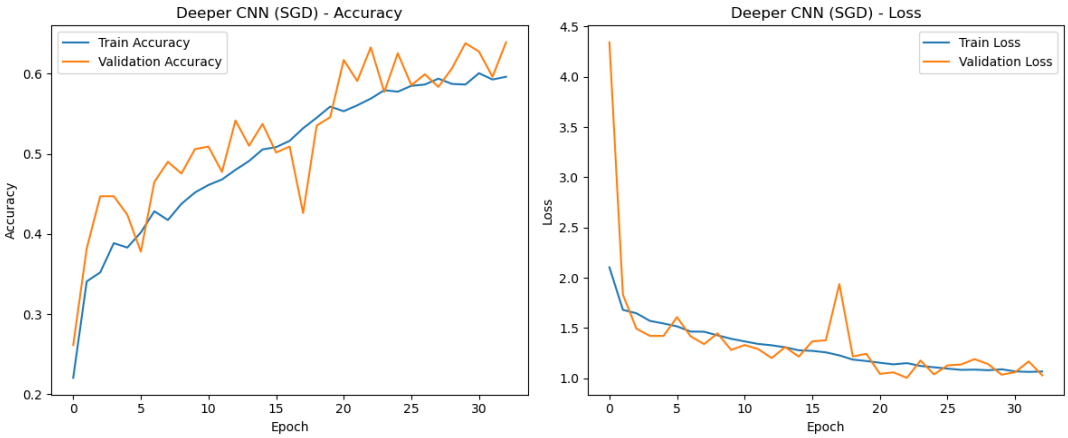

Figure 8: Accuracy and loss curves for the deeper CNN trained with SGD.

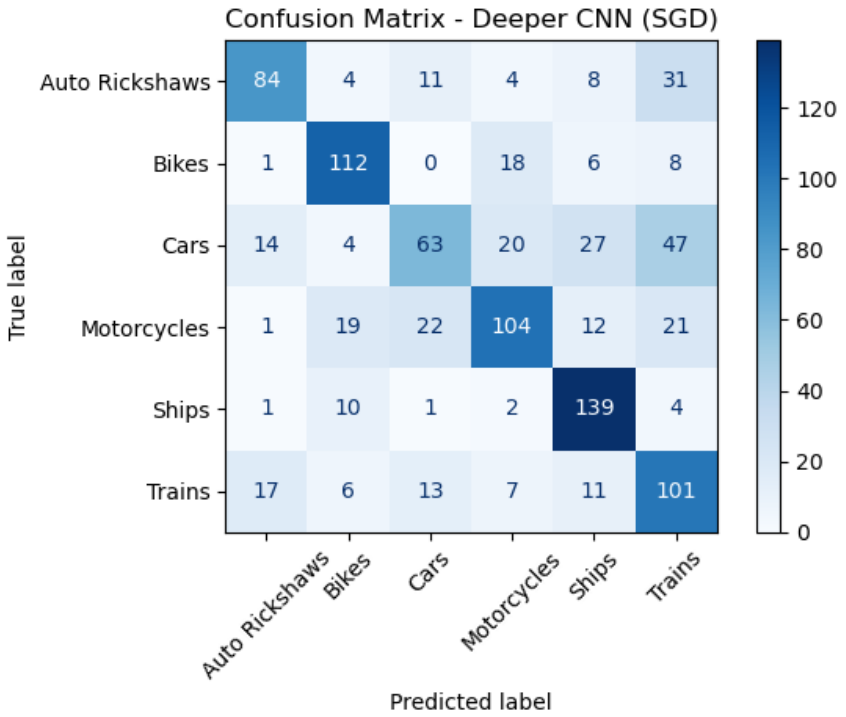

Figure 9: Confusion matrix for the deeper CNN trained with SGD.

The transfer learning model using MobileNetV2 as a frozen feature extractor reached 63.78% accuracy. Although this was lower than the custom CNN models, it still showed that pre-trained features can provide a useful starting point. When the model was fine-tuned, performance improved to 77.30%, which was close to the baseline CNN while requiring less training time. This confirms that fine-tuning can help a pre-trained model adapt to a new dataset.

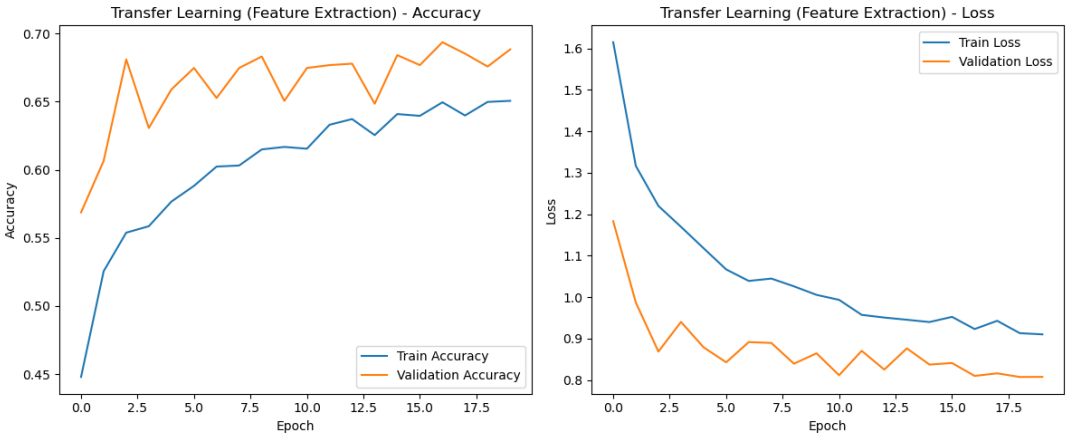

Figure 10: Accuracy and loss curves for transfer learning.

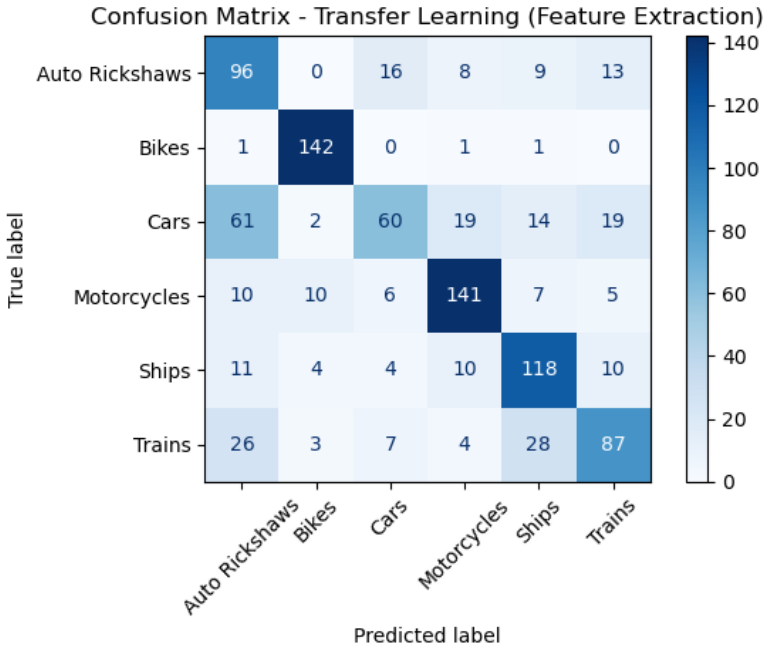

Figure 11: Confusion matrix for transfer learning.

The model comparison output summarises accuracy, precision, recall, F1-score and training time for all models. The baseline CNN had the best accuracy and F1-score, while fine-tuned transfer learning achieved similar performance in a shorter training period. The deeper CNN models were useful for comparison, but they required significantly longer training time, especially in the CPU-only environment.

Model Performance Comparison:

	Model	Accuracy	Precision	Recall	F1-Score	Training Time (min)
0	Baseline CNN	0.7820	0.7850	0.7820	0.7769	20.12
1	Deeper CNN (Adam)	0.7667	0.7988	0.7667	0.7669	158.29
2	Deeper CNN (SGD)	0.7252	0.7424	0.7252	0.7195	125.53
3	Transfer Learning	0.6378	0.6466	0.6378	0.6297	30.25
4	Fine-tuned Transfer Learning	0.7730	0.7744	0.7730	0.7710	26.40

Figure 12: Model performance comparison table generated from the code.

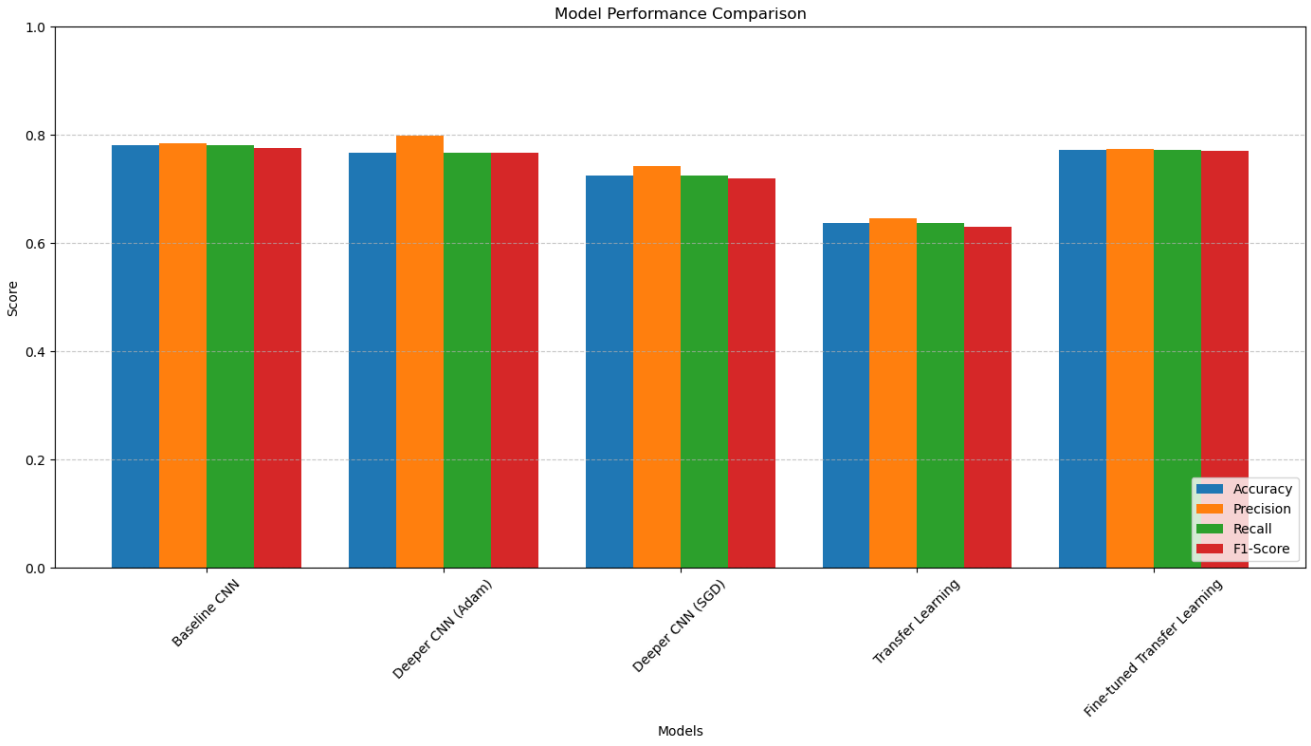

Figure 13: Bar chart comparison of model accuracy, precision, recall and F1-score.

Discussion

The results show that a larger or deeper model is not always the best model. The baseline CNN performed strongly because it had enough capacity to learn vehicle features without becoming too complex. The deeper CNN introduced Batch Normalization and Dropout, which helped control overfitting, but it also increased training time. In a CPU-only environment, this made experimentation slower and less efficient.

The comparison between Adam and SGD shows that Adam was more suitable for this experiment. Adam reached better performance faster because it adjusts the learning rate during training. SGD can still be useful, but it usually needs careful tuning and more training time. The transfer learning results also show that using a pre-trained model is helpful, especially after fine-tuning. Fine-tuning allowed MobileNetV2 to learn features that were more specific to the vehicle dataset.

Challenges in Training

The main challenge was hardware limitation. The models were trained in a CPU-only environment, which caused long training times and limited the size of experiments that could be tested. The baseline CNN took around 20 minutes, while the deeper CNN models took much longer. This shows that deep learning experiments benefit strongly from GPU or TPU acceleration.

Another challenge was overfitting. The deeper CNN had more parameters, which increased the risk of learning the training images too closely instead of learning general patterns. The code addressed this problem through Dropout, Batch Normalization and strong augmentation. Data quality was also a challenge because corrupted or unsuitable images needed to be removed so that the model could train without errors.

Conclusion and Future Work

This assignment successfully implemented and compared multiple CNN-based approaches for vehicle image classification. The baseline CNN produced the highest reported accuracy of 78.20%, while the fine-tuned MobileNetV2 model achieved a very similar result of 77.30% with a shorter training time. The deeper CNN models showed the effect of additional layers, regularization and

optimizer choice, but they also demonstrated that more complex models can be slower and may not always perform better.

In future work, the project could be improved by training with GPU or TPU support, using a larger and more diverse dataset and testing stronger architectures such as ResNet or EfficientNet. The model could also be improved through better hyperparameter tuning, learning-rate scheduling and explainability methods such as Grad-CAM. These improvements would make the system more reliable and easier to interpret in real-world vehicle recognition applications.

References

The report content is based on the submitted assignment document and its included code outputs, charts, confusion matrices and model comparison figures. The dataset source mentioned in the assignment is Kaggle, and the deep learning implementation follows common TensorFlow and Keras CNN and transfer learning workflows.